

/* This note has been compiled from various sources and can be taken as reference only*/

Chapter 4: I/O and Memory Management

Principles of I/O Hardware

I/O Devices

Input/output device is used to enter information and instructions into a computer for storage or processing and to deliver the processed data to a human operator or, in some cases, a machine controlled by the computer. It is also known as computer peripheral. I/O devices can be roughly divided into two categories:

- Block devices and
- Character devices

⇒ Block devices:

A device which stores information in fixed size blocks, each one with its own address is termed as block device. Common block size would be range from 512 bytes to 32,768 bytes. A block device can be read or write independently of all the other ones. Example: Disks are the most common block devices.

⇒ Character devices:

A character device delivers or accepts a stream of characters, without regard to any block structure. It is not addressable and does not have any seek operation. Example: printer, mouse, etc

Device Controllers

- An electronic device in the form of chip or circuit board that controls functioning of the I/O device is called the *device controller* or *I/O Controller* or *adopter*. The operating system directly deals with the device controller.
- It is the hardware that controls the communication between the system and the peripheral drive unit.
- Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller.
- Each device controller has a local buffer and a command register. It communicates with the CPU by interrupts.
- A device's controller plays an important role in the operation of that device; it functions as a bridge between the device and the operating system.

- ❑ I-O devices generally contain two parts: *mechanical* and *electrical* part. This electrical part is known as a device controller and can take the form of a chip on personal computers and mechanical part is a device.
- ❑ It takes care of low level operations such as error checking, moving disk heads, data transfer, and location of data on the device.
- ❑ There are many device controllers in a computer system. Example: serial port controller, SCSI bus controller, disk controller

Function of device controllers:

- ❑ Stops and starts the activity of the peripheral device.
- ❑ Generate error checking code.
- ❑ Checks the error in the data received from the interface.
- ❑ Abort that command which have errors.
- ❑ Retry the command having an error
- ❑ Receives the control signals from the interface unit
- ❑ Convert the format of the data
- ❑ Check the status of the device.

Memory Mapped I/O and I/O Mapped I/O

⇒ Memory Mapped I/O

Memory Mapped I/O is a technique in which an I/O device is mapped as memory itself. In this approach, memory and I/O devices occupy the same space as far as the system is concerned. This implies that the CPU can transmits or receive data from I/O devices using the same format that is used to address normal memory.

Characteristics of Memory Mapped I/O

- **Shared Address Space:** I/O devices are carved out a particular memory space.
- **Unified Instructions:** They are identical for both memory and I/O operations.
- **Simplified Addressing** This means there is no need for special instructions and port for I/O operations as in integrated I/O.

Advantages of Memory Mapped I/O

- **Faster Access:** Because a memory has faster access times then an I/O operation can have improved access times as well.

- **Flexible Addressing:** Supports all the addressing modes of the CPU hence making it easier to write programs.
- **Simplified Programming Model:** Unlike other instruction types, where the Developer requires a special kind of instruction called I/O instruction, they are free to use what is called memory instruction.

Disadvantages of Memory Mapped I/O

- **Limited Address Space:** It was also pointed out that sharing the address space between memory and I/O can reduce the number of available memory addresses.
- **Complexity in Address Decoding:** needs extra software logic in order to distinguish between memory accesses and I/O operations.
- **Potential for Address Conflicts:** Possibility of the existence of similar addresses if not controlled properly.

⇒ I/O Mapped I/O?

I/O Mapped I/O known as Port Mapped I/O uses dedicated address space for the installation of I/O devices. This method employs specific port commands by which it addresses I/O ports hence enhancing the segregation between memory and ports.

Features of I/O Mapped I/O

- **Separate Address Space:** I/O devices are granted their own address space that is independent of the memory space of the computer.
- **Dedicated Instructions:** Contains specific I/O instructions known as `IN` and `OUT`.
- **Distinct Port Addresses:** I/O devices are connected to the computer and can only be addressed by specific port addresses.
-

Advantages of I/O Mapped I/O

- **Expanded Address Space:** Preserves memory address space full for the memory operations use, to avoid any conflict.
- **Simplified Address Decoding:** Hence, it lowers the ontological level seen in previous models and clears the memory –I/O barrier making the model less complex.
- **Security and Stability:** decreases the possibility of an authority or a code somehow getting the memory's address and accessing it through I/O channels.

Disadvantages of I/O Mapped I/O

- **Slower Access:** I/O operations may be slower because of the use of special instruction.

- **Limited Addressing Modes:** Limited to the addressing modes of I/O instructions; thus the flexibility is somewhat hampered.
- **Increased Instruction Set:** Needs further instructions which adds to the intricate instruction set architecture.

DMA (Direct Memory Access)

DMA Stands for "Direct Memory Access." DMA is a method of transferring data from the computer's RAM to another part of the computer without processing it using the CPU. While most data that is input or output from your computer is processed by the CPU, some data does not require processing, or can be processed by another device. In these situations, DMA can save processing time and is a more efficient way to move data from the computer's memory to other devices.

In other words, direct memory access (DMA) is a method that allows an input/output (I/O) device to send or receive data directly to or from the main memory, bypassing the CPU to speed up memory operations. The process is managed by a chip known as a DMA controller (DMAC).

For example, a sound card may need to access data stored in the computer's RAM, but since it can process the data itself, it may use DMA to bypass the CPU. Video cards that support DMA can also access the system memory and process graphics without needing the CPU.

DMA is an essential feature of all modern computers, as it allows devices to transfer data without subjecting the CPU to a heavy overhead. With DMA, the CPU gets freed from this overhead and can do useful tasks during data transfer.

A DMA Controller is a device, which takes over the system bus to directly transfer information from one part of the system to another. The DMA controller is told to make a transfer either by the CPU or some special circumstances; then the DMA controller makes a request to gain control of the bus from the CPU, other processors, or controllers which might currently be using bus; these other devices then relinquish control of the bus by putting their lines into tri-state condition; they then grant the bus to the DMA controller; and finally, the DMA controller takes over the bus, generating its own address and control signals for the bus and causing the transfer of information.

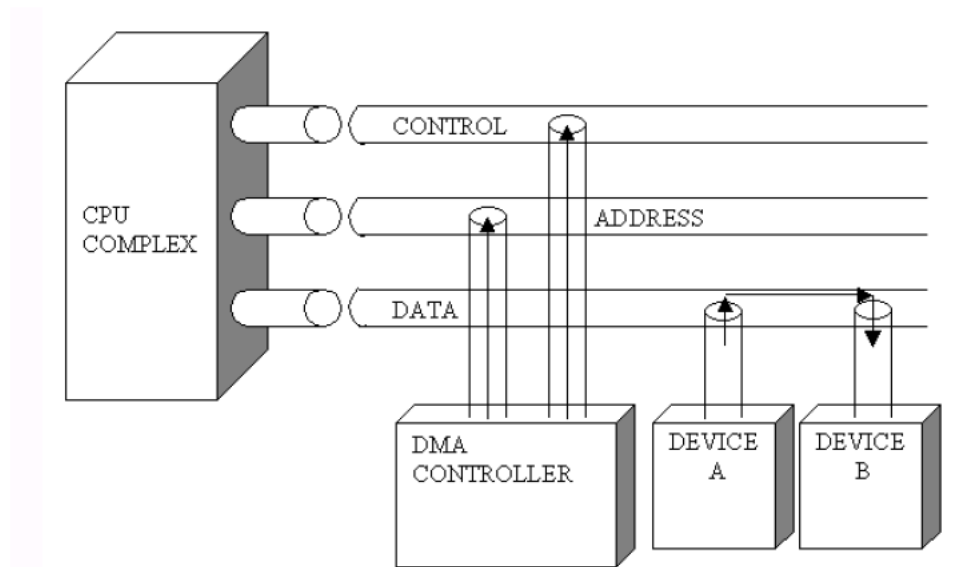


Figure 1: DMA transfer

Working of DMA

- ❑ Transfers a block of data directly to or from memory
- ❑ An interrupt is sent when the task is complete
- ❑ The processor is only involved at the beginning and end of the transfer

Principles of I/O Software

The basic idea of I/O software is to organize the software as a series of a layer. The lower layer contains the information of the hardware and the upper layer contains the interface to the user. The main concept of I/O software is called as device independence which that the program can be written or match in such a way that it can be read on hard disk without modifying the program for each different deice.

The main important job for I/O software is error handling; it controls the error by using control program (error handling).

The concept of I/O software is sharable and dedicated device. Some I/O devices such as hard disk can be used by multiple users at a same time. No problem is occurring even if multiple user open one particular file at one time.

Polled I/O and Interrupt driven I/O

Most input and output devices are much slower than the CPU—so much slower that it would be a waste of the CPU to make it wait for the input devices. There are two mechanisms: **Polling** and **Interrupt driven I/O**

⇒ Pooling:

Polling means determining the status of an I/O device with direct action taken by the CPU i.e. the mechanism for a CPU to check on the status of an I/O device is by reading a memory address which is associated with an I/O device (using memory mapped I/O) is called *polling*. To poll an I/O device, the CPU typically uses memory-mapped I/O. I/O devices are assigned memory addresses which aren't used by memory at all, but are instead, special addresses that the I/O device recognizes. The CPU can then check the status of the device by reading a byte or word at some pre-determined address for that particular I/O device. Occasionally, polling is the right thing to do, especially if devices are quick enough.

- The main advantage of polling is that the CPU determines how often it needs to poll.
- The problem with polling is that the CPU operates at a much faster speed than most I/O devices. Thus, a CPU can get into a busy wait, checking the device many times, even though the device is very slow.

Example:

Let us consider a game where a basketball player is asked to make as many free-throws as possible in one minute, but the clock is down the hall in another room. The player must run down the hall and check if the minute has passed, and if not, go back to the gym and try to make another shot, then run down the hall to check the clock and run back to take another shot. The player spends much of the time simply checking (polling) the clock.

⇒ Interrupts

An interrupt causes the CPU to "stop" and determine what device is interrupting. Here the CPU works on its given tasks continuously. When an input is available, such as when someone types a key on the keyboard, then the CPU is interrupted from its work to take care of the input data. The CPU can work continuously on a task without checking the input devices, allowing the devices themselves to interrupt it as necessary. This requires some extra "smarts" in the form of electronic circuitry at the I/O devices so that they can interrupt the CPU.

Goals of I/O Software

The main goal of I/O Software has been broadly classified into four layers:

- Interrupt Handler
- Device Driver
- Device dependent operating system software

□ User level interface

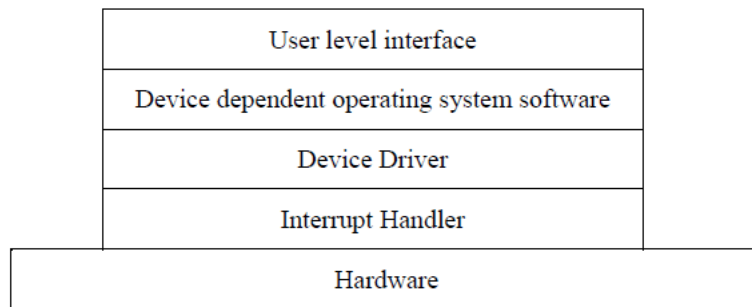


Fig: Layer of I/O software system

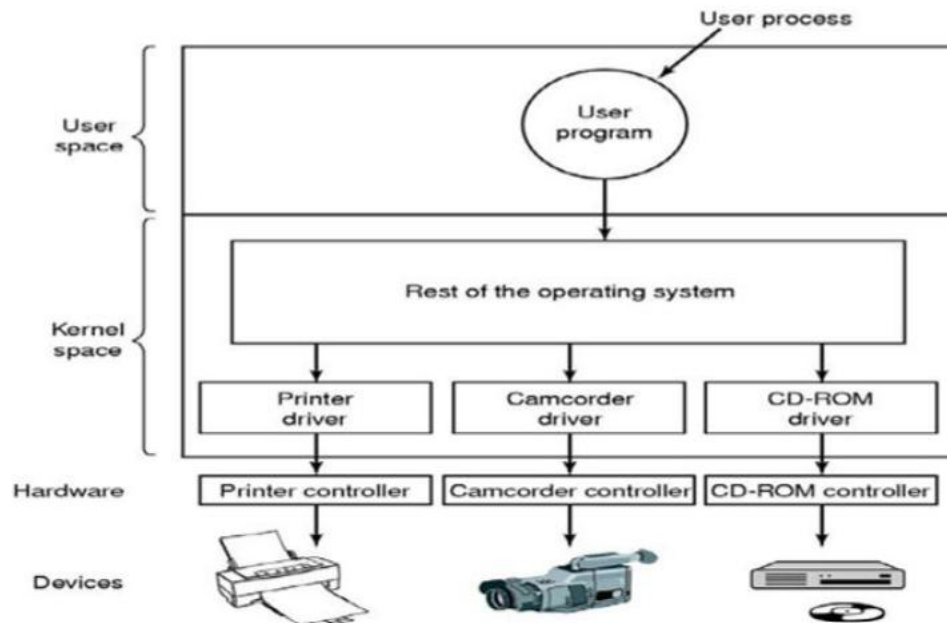
1. Interrupt Handler

It is the bottom layer of I/O software. When an event occurs the micro-controller generates a *hardware interrupt*. The interrupt forces the micro-controller's program counter to jump to a specific address in program memory. This special memory address is called the *interrupt vector*. At this memory location we install a special function known as an *interrupt service routine (ISR)* which is also known as an *interrupt handler*. So when a hardware interrupt is generated, program execution jumps to the interrupt handler and executes the code in that handler. When the handler is done, then program control returns the micro-controller to the original program it was executing. So a hardware interrupt allows a micro-controller to interrupt an existing program and react to some external hardware event.

2. Device Driver

A **device driver** is a computer program that operates or controls a particular type of device that is attached to a computer. A driver provides a software interface to hardware devices, enabling operating systems and other computer programs to access hardware functions without needing to know precise details of the hardware being used.

A driver is a small piece of software that tells the operating system and other software how to communicate with a piece of hardware.



3. Device independent operating system software

Device independence is the process of making a software application is able to function on a wide variety of devices i.e. the capability of a program, operating system or programming language to work on varieties of computers or peripherals, despite their electronic variation.

Function of device-independent I/O software:

- ☐ Uniform interfacing for device drivers
- ☐ Buffering
- ☐ Error reporting
- ☐ Allocating and releasing dedicate devices
- ☐ Providing a device-independent block size

4. User level interface

It is the upper most layer of I/O system. This layer mainly represents the outermost user interface that occurs between the operating system and end user. A user interface is that portion of an interactive computer system that communicates with the user. Proper design of a user interface can make a significant difference in training time, performance speed, error rates, user satisfaction, and the user's

retention of knowledge of operations over time.

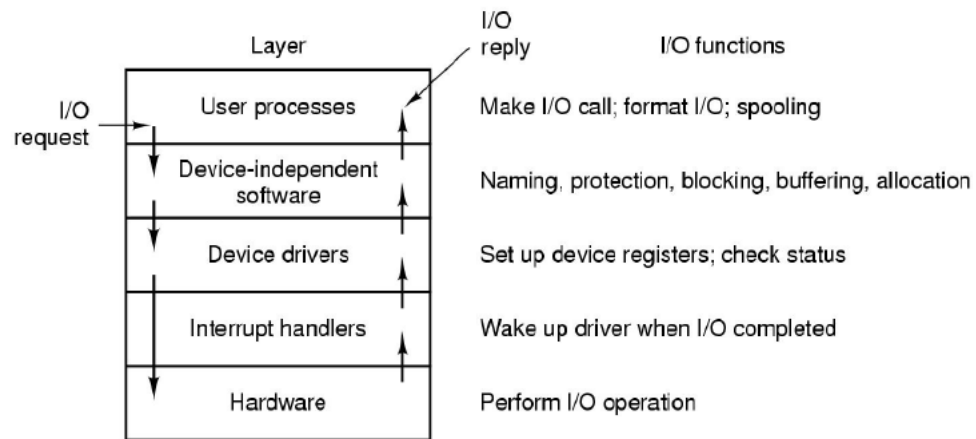


Fig: Layer of I/O System

Disk Technologies

1. Magnetic Disks (HDD – Hard Disk Drives)

Definition: Magnetic disks are traditional data storage devices that use magnetic storage to read and write data using spinning platters and read/write heads.

Key Features:

- **Structure:** Consist of one or more platters coated with magnetic material.
- **Read/Write Head:** Floats above the disk surface to access data.
- **Rotational Speed:** Typically 5400 to 7200 RPM in consumer drives (up to 15,000 RPM in enterprise drives).
- **Access Time:** Slower due to mechanical movement.
- **Data Transfer:** Sequential data access is faster than random access.

Pros:

- Cost-effective for large storage.
- Good for archiving and less frequently accessed data.

Cons:

- Slower compared to SSDs.
- Mechanical parts prone to wear and failure.
- Heavier and consumes more power.

◆ SATA vs. PATA

☑ 1. PATA (Parallel ATA)

Full Form: Parallel Advanced Technology Attachment

Also known as: IDE (Integrated Drive Electronics)

Used in: Older hard drives and optical drives (up to early 2000s)

Key Features:

- **Parallel data transfer** (multiple bits at once)
- **40-pin or 80-wire ribbon cable**

- **Data Transfer Rate:** Up to 133 MB/s (ATA/133)
- **Master/Slave** configuration for two devices per cable
- **Short cable length** (typically 18 inches / 46 cm)

Limitations:

- Bulky cables (reduce airflow inside PCs)
- Lower speed and reliability compared to SATA
- Obsolete in modern systems

☑ 2. SATA (Serial ATA)

Full Form: Serial Advanced Technology Attachment

Introduced: Early 2000s (to replace PATA)

Key Features:

- **Serial data transfer** (1 bit at a time, but much faster)
- **7-pin cable** for data + separate power connector
- **Data Transfer Rates:**
 - SATA I: 1.5 Gbps (~150 MB/s)
 - SATA II: 3.0 Gbps (~300 MB/s)
 - SATA III: 6.0 Gbps (~600 MB/s)
- **Hot-swappable** (can plug/unplug while powered on)
- **One device per cable** (no master/slave needed)
- **Longer cables** (up to 1 meter)

← END Comparison

Feature	PATA (IDE)	SATA
Data Transfer	Parallel	Serial
Max Speed	~133 MB/s	Up to ~600 MB/s (SATA III)
Cable Type	40/80-wire ribbon Thin 7-pin cable	

Feature	PATA (IDE)	SATA
Devices per Cable	2 (Master/Slave)	1
Hot Swapping	☐ No	☒ Yes
Use in Modern PCs	☐ Obsolete	☒ Standard (until NVMe)

2. What is an SSD?

SSD (Solid-State Drive) is a type of **non-volatile storage device** that stores data on **flash memory** (usually NAND-based), with **no moving parts**.

Key Features:

- Much **faster** than HDDs
- **More durable** (shock-resistant)
- **Consumes less power**
- Commonly used in laptops, desktops, and servers

SSD Types (Based on Interface):

1. **SATA SSD** – Uses the **SATA interface**, slower (~550 MB/s max)
2. **NVMe SSD** – Uses the **NVMe protocol** over **PCIe interface**, much faster (up to 7000+ MB/s)

◆ What is NVMe?

NVMe (Non-Volatile Memory Express) is a **communication protocol** developed specifically for SSDs that use the **PCIe interface**.

Key Features:

- Designed for **flash memory**, unlike SATA (designed for HDDs)
- **High-speed** data transfer (up to 5x–10x faster than SATA SSD)
- **Supports parallelism**: 64,000 queues × 64,000 commands
- **Lower latency**, ideal for gaming, content creation, servers, etc.

Relationship Between SSD and NVMe:

Concept	Description
SSD	A type of storage drive using flash memory
NVMe	A protocol used by some SSDs for faster communication over PCIe
NVMe SSD	An SSD that uses the NVMe protocol over PCIe for ultra-fast performance
SATA SSD	An SSD that uses the SATA interface , slower but widely compatible

- ✓ All NVMe drives are SSDs, but
- ✗ Not all SSDs are NVMe — some are SATA-based.

Summary Table:

Feature	SATA SSD	NVMe SSD
Interface	SATA (AHCI protocol)	PCIe (NVMe protocol)
Max Speed	~550 MB/s	3500–7000+ MB/s
Protocol Designed For	HDD (originally)	SSD (flash memory)
Latency	Higher	Very Low
Cost	Lower	Higher
Use Case	Everyday computing	High-performance apps

What is RAID?

RAID (Redundant Array of Independent(Inexpensive) Disks) is a technology that combines multiple physical drives into a single logical unit to enhance **performance, fault tolerance**, or both RAID can be implemented via:

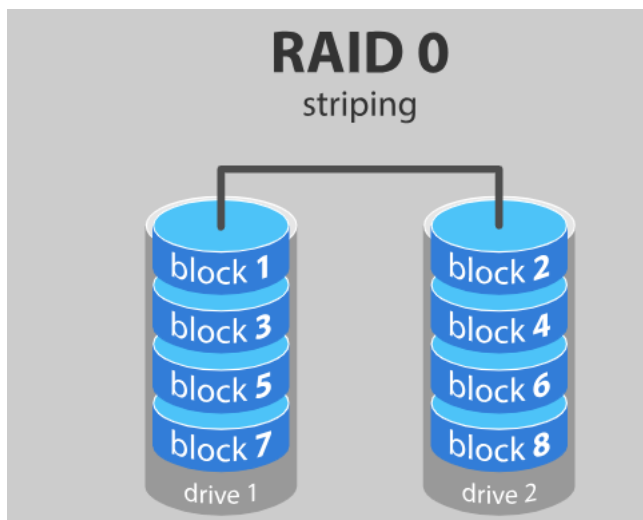
- **Hardware RAID:** Managed by dedicated controllers (PCIe cards or on-chip).
- **Software RAID:** Handled by the OS

◆ RAID Levels

RAID 0 – Striping (no redundancy)

In a RAID 0 system data are split up into blocks that get written across all the drives in the array. By using multiple disks (at least 2) at the same time, this offers superior I/O performance. This performance can be enhanced further by using multiple controllers,

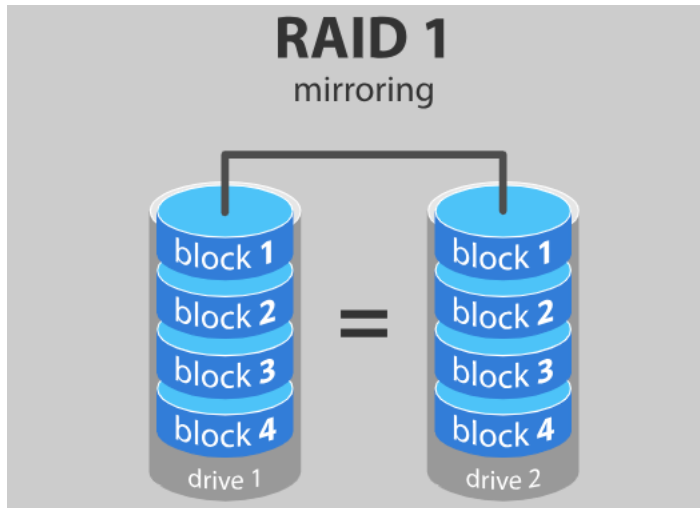
ideally one controller per disk.



- **How it works:** Splits data across ≥ 2 disks for parallel access.
- **Pros:** Highest read/write speed; full storage capacity
- **Cons:** No fault tolerance—single disk failure = total data loss
- **Use case:** High-speed temporary data (e.g., video editing).

RAID 1 – Mirroring

Data are stored twice by writing them to both the data drive (or set of data drives) and a mirror drive (or set of drives). If a drive fails, the controller uses either the data drive or the mirror drive for data recovery and continuous operation. You need at least 2 drives for a RAID 1 array.



- **How it works:** Writes identical data to ≥ 2 disks.
- **Pros:** Data survives if one disk fails; faster reads
- **Cons:** 50% storage efficiency; write speed limited by slowest disk
- **Use case:** Critical systems requiring reliability (e.g., OS drives).

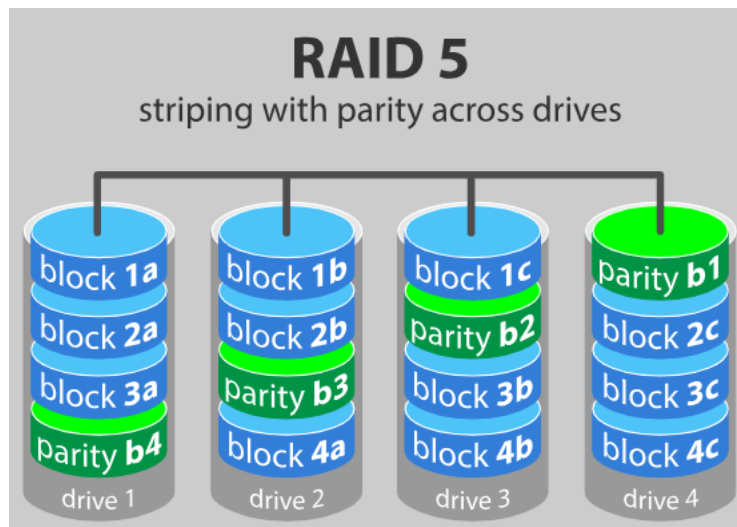
RAID 2 Obsolete,

RAID3 Striping with parity in single disk (overhead on disk with parity in each write operation)

RAID 4 Similar to 3

RAID 5 – Striping with Distributed Parity

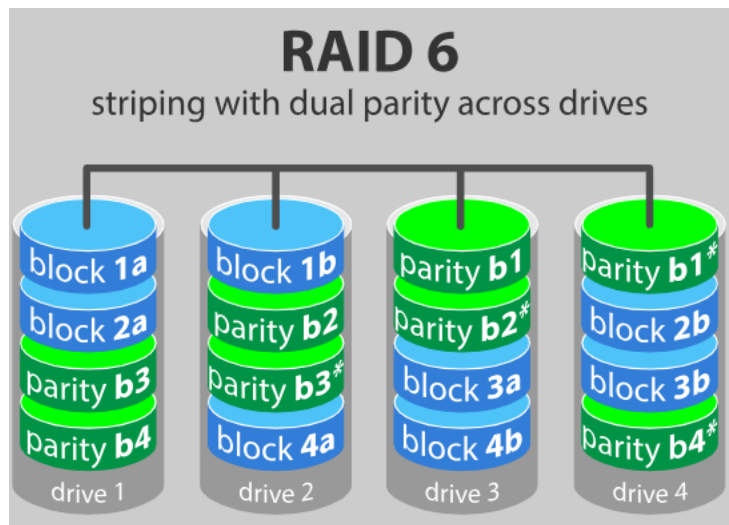
RAID 5 is the most common secure RAID level. It requires at least 3 drives but can work with up to 16. Data blocks are striped across the drives and on one drive a parity checksum of all the block data is written. The parity data are not written to a fixed drive, they are spread across all drives, as the drawing below shows. Using the parity data, the computer can recalculate the data of one of the other data blocks, should those data no longer be available. That means a RAID 5 array can withstand a single drive failure without losing data or access to data. Although RAID 5 can be achieved in software, a hardware controller is recommended. Often extra cache memory is used on these controllers to improve the write performance.



- **How it works:** Data and parity blocks striped across ≥ 3 disks.
 - **Pros:** Balanced performance, fault tolerance (1 disk), and storage efficiency
 - **Cons:** Write operations slowed by parity calculations; rebuild time can impact performance .
 - **Use case:** General-purpose servers and file storage.
-

RAID 6 – Striping with Double Parity

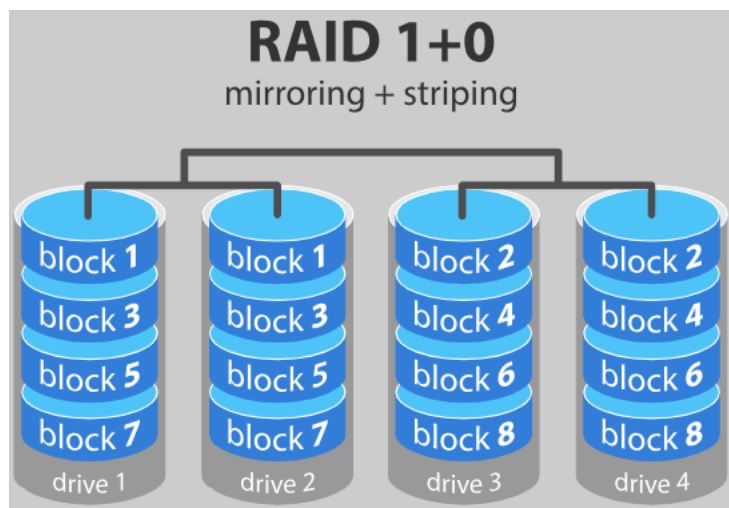
RAID 6 is like RAID 5, but the parity data are written to two drives. That means it requires at least 4 drives and can withstand 2 drives dying simultaneously. The chances that two drives break down at exactly the same moment are of course very small. However, if a drive in a RAID 5 systems dies and is replaced by a new drive, it takes hours or even more than a day to rebuild the swapped drive. If another drive dies during that time, you still lose all of your data. With RAID 6, the RAID array will even survive that second failure.



- **How it works:** Like RAID 5 but with two parity blocks (blocks P and Q)
- **Pros:** Tolerates up to 2 disk failures; high data protection
- **Cons:** Higher overhead; slower writes due to dual parity
- **Use case:** High-availability systems (e.g., large databases).

RAID 10 (1+0) – Nested (Mirroring + Striping)

It is possible to combine the advantages (and disadvantages) of RAID 0 and RAID 1 in one single system. This is a nested or hybrid RAID configuration. It provides security by mirroring all data on secondary drives while using striping across each set of drives to speed up data transfers.



- **How it works:** Combines RAID 1 mirrors across striped sets; requires ≥ 4 disks
- **Pros:** High performance, strong redundancy—even if one disk per mirrored pair fails
- **Cons:** Only 50% usable capacity; expensive .
- **Use case:** Performance-critical, fault-tolerant environments (e.g., high-transaction databases).

Summary Table

RAID Level	Min Disks	Storage Efficiency	Fault Tolerance	Performance
RAID 0	2	100%	✗ None	 Very High
RAID 1	2	50%	☑ 1 disk	 Good reads, slower writes
RAID 5	3	$(N-1)/N$	☑ 1 disk	 Balanced
RAID 6	4	$(N-2)/N$	☑ 2 disks	 Read fast, writes slower
RAID 10	4	50%	☑ 1 per mirror pair	 High performance

Implementation Types

- **Hardware RAID:** Higher performance, less CPU overhead, OS-transparent
- **Software RAID:** Lower cost, flexible, but depends on CPU and OS support

Stable storage

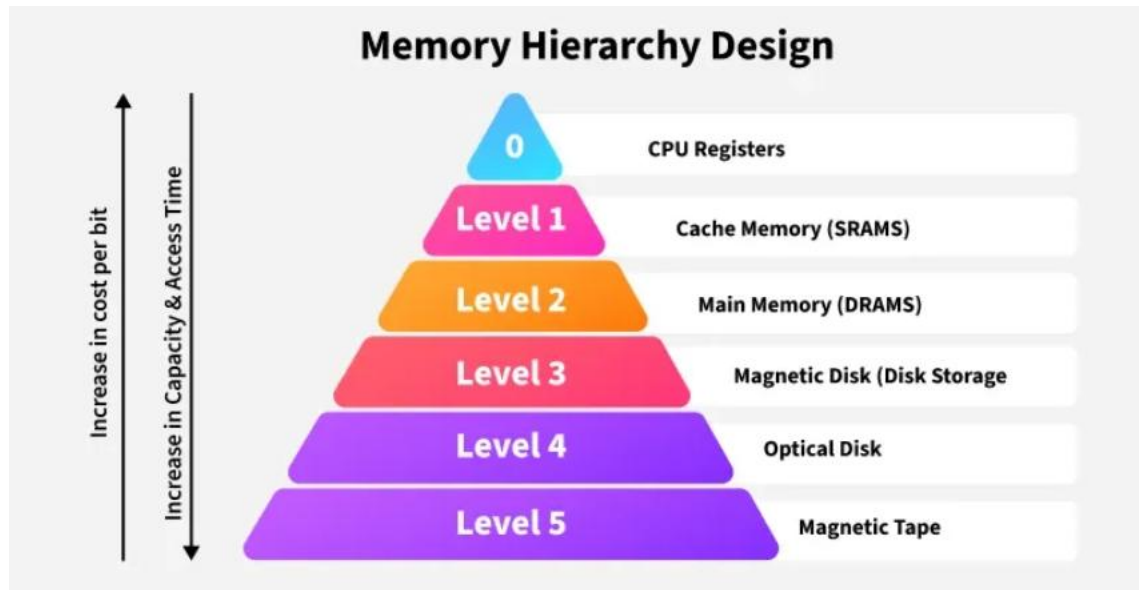
Stable Storage is a **theoretical model** of storage in which **data is never lost or corrupted**, even in the case of:

- Power failures
- System crashes
- Hardware faults

Achieved by techniques like:

- **Redundant copies** of data
- **Checksums** to detect corruption
- **Atomic writes** (write either completes fully or not at all)
- **Write-ahead logging** and **journaling** (e.g., in file systems like ext4, NTFS)

Cost per bit comparison of memory



#Numerical on cost per bit

Calculate the average cost per bit of a system with the main memory of 1024 cost 12 units/bit and secondary memory of 4096 bits 10 units/bit

- per bit cost of main memory will be 12
- per bit cost of secondary memory will be 10
- Now average cost per bit will be = Total cost of system / total number of bits.
$$= (1024 \times 12) + (4096 \times 10) / (1024 + 4096)$$
$$= (12,288 + 40960) / (1024 + 4096)$$
$$= 53,248 / 5,120$$
$$= 10.4 \text{ units}$$

Memory Management

Memory is the physical device which is used to store programs or data on a temporary or permanent basis for use in a computer or other digital electronics device. There are various types of memories in a computer system and are accessed by various processes for their execution.

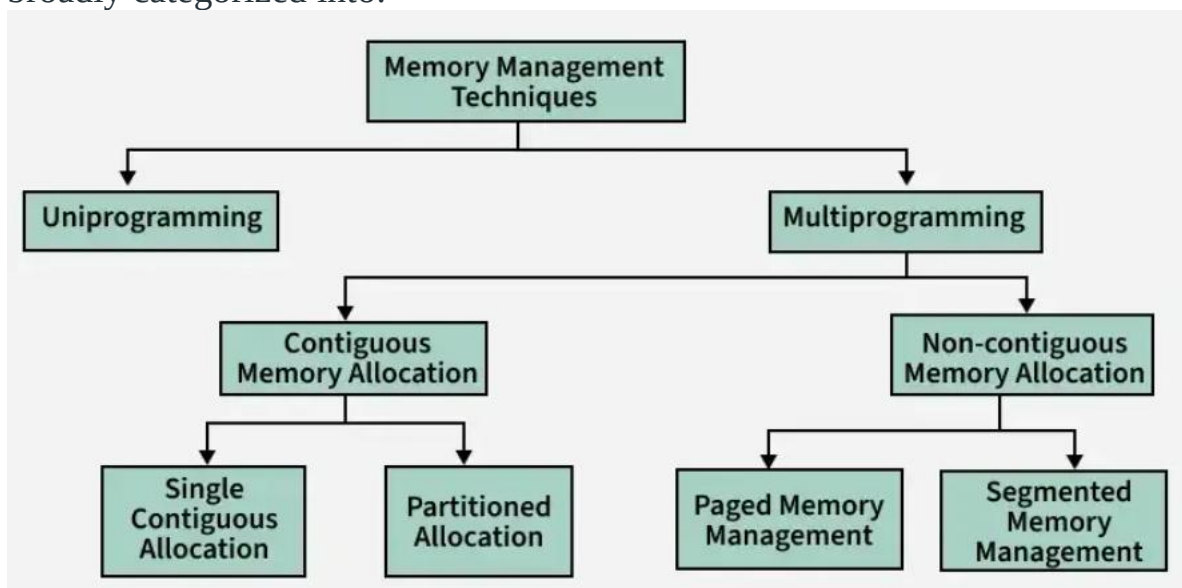
It is most important and most complex task of an operating system. **Memory management involves treating main memory as a resource to be allocated to and shared among number of active processes i.e. it is the act of managing computer memory. It involves providing ways to allocate portions of memory to programs at their request, and freeing it for reuse when no longer needed.** To efficiently use the processor and the I/O facilities, it is desirable to maintain as many processes in main memory as possible.

Memory management is sub module of an OS, whose responsibility is to

- ❑ Allocate and de-allocate memory space as requested.
- ❑ Keep track of which parts of memory are currently being used and by whom.
- ❑ Efficient utilization when the memory resource is heavily competed

Memory Management Techniques

Memory management techniques are methods used by an operating system to efficiently allocate, utilize, and manage memory resources for processes. Various techniques help the operating system manage memory effectively. They can be broadly categorized into:



Logical and Physical Address Space

- **Logical Address Space:** An address generated by the CPU is known as a “Logical Address”. It is also known as a Virtual address. Logical address space can be defined as the size of the process. A logical address can be changed.
- **Physical Address Space:** It refers to the set of actual addresses used by the memory hardware. A physical address, also called a real address, is generated by the Memory Management Unit (MMU) through run-time mapping of virtual addresses. Unlike virtual addresses, physical addresses remain constant.

Static and Dynamic Loading

Loading a process into the main memory is done by a loader. There are two different types of loading :

- **Static Loading:** Static Loading is basically loading the entire program into a fixed address. It requires more memory space.
- **Dynamic Loading:** Dynamic loading loads program routines into memory only when they are needed. This saves memory by not loading unused routines. The routines remain on disk in relocatable (can be loaded at any memory location) format until called. It allows better memory utilization, especially for large programs.

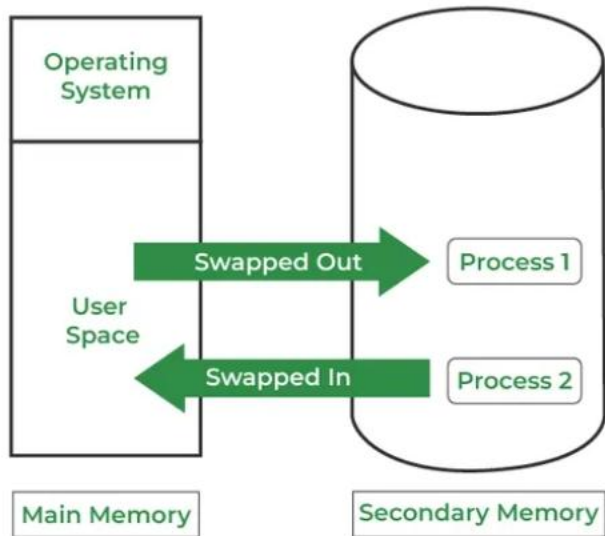
Static and Dynamic Linking

To perform a linking task a linker is used. A linker is a program that takes one or more object files generated by a compiler and combines them into a single executable file.

- **Static Linking:** In static linking, the linker combines all necessary program modules into a single executable program. So there is no runtime dependency. Some operating systems support only static linking, in which system language libraries are treated like any other object module.
- **Dynamic Linking:** The basic concept of dynamic linking is similar to dynamic loading. In dynamic linking, "Stub" is included for each appropriate library routine reference. A stub is a small piece of code. When the stub is executed, it checks whether the needed routine is already in memory or not. If not available then the program loads the routine into memory.

Swapping

Swapping moves processes between main memory and secondary memory to manage limited memory space. It allows multiple processes to run by temporarily swapping out lower priority processes for higher priority ones. The swapped-out process resumes once it's loaded back. Transfer time depends on the amount of data swapped.



Swapping

Non-Contiguous Memory Allocation

This method allows processes to be broken into smaller parts, which are placed in different, non-adjacent memory locations. Techniques for non-contiguous memory allocation include:

- **Paging:** The process is divided into fixed-size blocks called "pages," and the memory is divided into blocks of the same size called "frames." The operating system keeps a page table to map logical pages to physical frames.
- **Segmentation:** The process is divided into segments of varying sizes, such as code, data, stack, etc. The operating system maintains a segment table to map logical segments to physical memory.

Fragmentation

Fragmentation is defined as when the process is loaded and removed after execution from memory, it creates a small free hole. These holes can not be assigned to new processes because holes are not combined or do not fulfill the memory requirement of the process. In the operating systems two types of fragmentation are:

- **Internal fragmentation:** Happens when fixed-sized memory blocks are allocated to processes larger than needed, leaving unused space within the allocated block. For example, if a process needs 2MB of memory but is allocated a 3MB block, 1MB is wasted.
- **External fragmentation:** Occurs when free memory is scattered in small blocks across the system, making it impossible to allocate a large contiguous block to a process, even though the total free memory is enough.

Memory Allocation Strategies

Efficient memory allocation is essential for optimal performance. There are several strategies for allocating memory blocks:

Fixed Partition Allocation: Memory is divided into fixed-sized partitions, and each partition can hold only one process. The OS keeps track of free and occupied partitions using a partition table.

Dynamic Partition Allocation: Memory is divided into variable-sized partitions based on the size of the processes. This helps avoid wastage of memory but can result in fragmentation.

Placement Algorithms: When allocating memory, the OS uses placement algorithms to decide which free block should be assigned to a process:

- **First Fit:** Allocates the first available partition large enough to hold the process.
- **Best Fit:** Allocates the smallest available partition that fits the process, reducing wasted space.
- **Worst Fit:** Allocates the largest available partition, leaving the largest remaining space.
- **Next Fit:** Similar to First Fit but starts searching for free memory from the point of the last allocation.

For the following partition of 100K, 500K, 200K, 300K and 600K (in order) place the processes 212K, 417K, 122K, and 426K in order according to the: Best fit and worst fit

Soln: Here, the given processes are:

P1 = 212K P2 = 417K P3 = 122K P4 = 426K

i. Best fit

Operating System

	100K
P2	500K
P3	200K
P1	300K
P4	600K

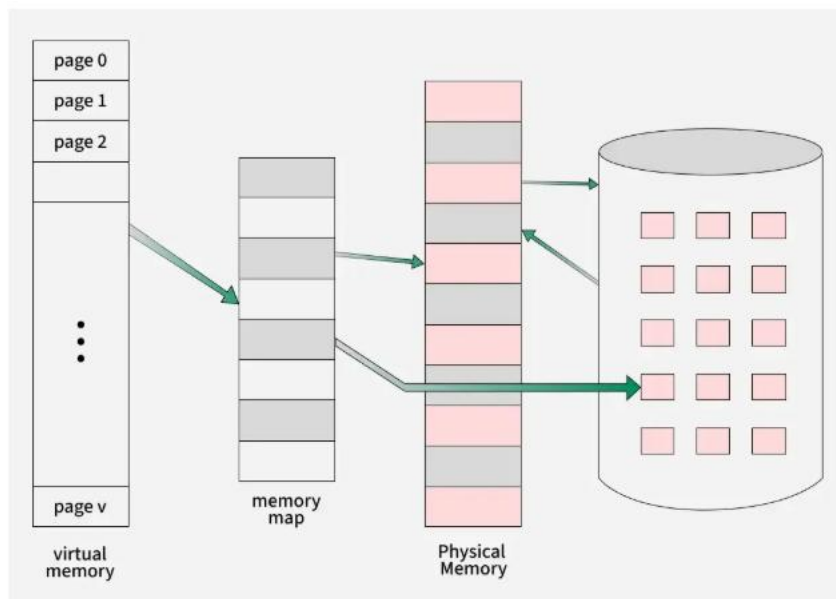
Fig: Best Fit

Operating System	
	100K
P2	500K
	200K
P3	300K
P1	600K

Fig: Worst Fit

Virtual Memory Management

The main memory is considered as the physical memory in which many programs want to reside. However, the size of such memory is limited and it cannot hold or load all the programs simultaneously. To overcome such types of problem, the overloaded inactive programs are swapped out from the main memory to hard disk for certain period of time. Such technique is known as virtual memory i.e. **Virtual memory** is a memory management system in a computer that temporarily stores inactive parts of the content of RAM on a disk, restoring it to RAM when quick access to it is needed.



Types of Virtual Memory

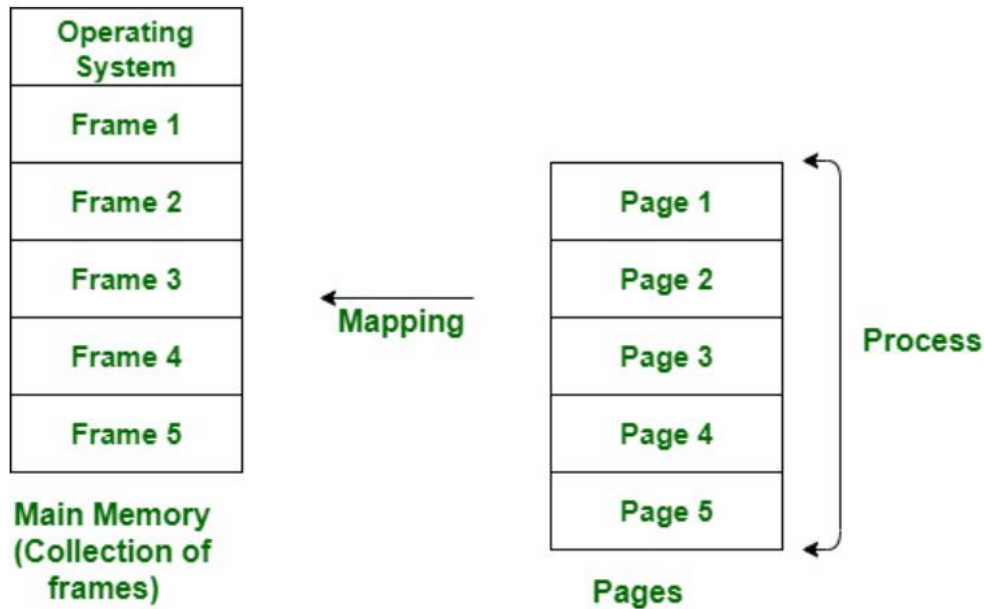
In a computer, virtual memory is managed by the Memory Management Unit (MMU), which is often built into the CPU. The CPU generates virtual addresses that the MMU translates into physical addresses.

There are two main types of virtual memory management:

- Paging
- Segmentation

Paging

Paging divides memory into small fixed-size blocks called pages. When the computer runs out of RAM, pages that aren't currently in use are moved to the hard drive, into an area called a swap file. The swap file acts as an extension of RAM. When a page is needed again, it is swapped back into RAM, a process known as page swapping. This ensures that the operating system (OS) and applications have enough memory to run.



Demand Paging: The process of loading the page into memory on demand (whenever a page fault occurs) is known as demand paging.

Page table:

A **page table** is the data structure used by a virtual memory system in a computer operating system to store the mapping between virtual address and physical address i.e. it is used to map virtual page number to physical page number.

Page vs Frame

A page is a fixed size block of data in virtual memory and a frame is a fixed size block of physical memory in RAM where these pages are loaded.

Page Fault

The main functions of paging are performed when a program tries to access pages that are not currently mapped to physical memory (RAM). This situation is known as a **page fault**.

Thrashing

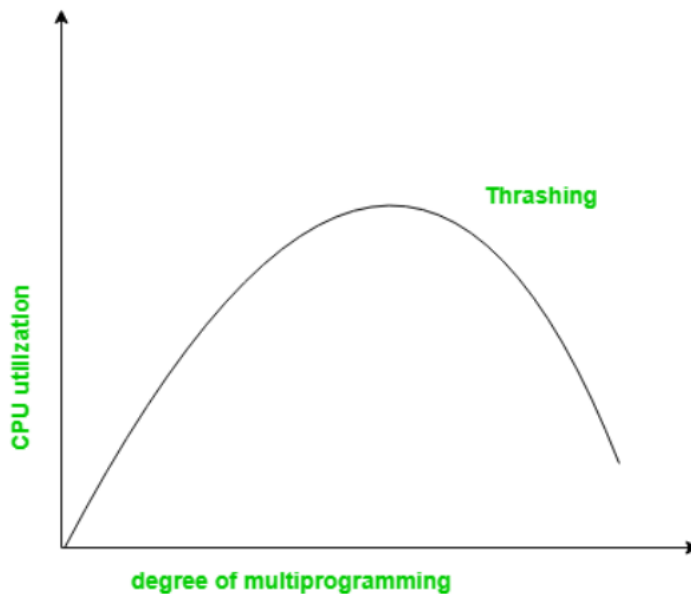
Thrashing is a condition in which excessive paging operations are taking place i.e. it refers to any situation in which multiple processes are competing for the same resource, and the excessive swapping back and forth between connections causes a slowdown. This causes the performance of the computer to degrade or collapse.

Thrashing is often caused:

- when the system does not have enough memory,
- the system swap file is not properly configured, or
- too much is running on the computer and it has low system resources.

To resolve hard drive thrashing, a user can do any of the below.

- ☐ Increase the amount of RAM in the computer.
- ☐ Decrease the amount of programs being run on the computer.
- ☐ Adjust the size of the swap file.



Page Replacement Algorithm

When a page fault occurs, the operating system has to choose a page to remove from memory to make room for the page that has to be brought in. The page replacement is done by swapping the required pages from backup storage to main memory and vice-versa. A page replacement algorithm is evaluated by running the particular algorithm on a string of memory references and computes the page faults.

The basic page replacement method:

1. Find the location of the desired page on the disk.
2. Find a free frame:
 - ☐ If there is a free frame, use it.
 - ☐ If there is no free frame, use a page replacement algorithm to select a victim frame.
 - ☐ Write the victim frame to the disk; change the page and frame tables accordingly.

3. Read the desired page into the newly free frame; change the page and frame tables.
4. Restart the user process.

1. First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.

Page reference		1, 3, 0, 3, 5, 6, 3					
1	3	0	3	5	6	3	
		0	0	0	0	3	
	3	3	3	3	6	6	
1	1	1	1	5	5	5	
Miss	Miss	Miss	Hit	Miss	Miss	Miss	

Total Page Fault = 6

FIFO - Page Replacement

2. Least Recently Used

In this algorithm, page will be replaced which is least recently used.

Example Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using LRU Page Replacement Algorithm.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

Least Recently Used - Page Replacement

3. Optimal Page Replacement Algorithm

Optimal Page Replacement is one of the Algorithms of Page Replacement. In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

The idea is simple, for every reference we do following:

1. If referred page is already present, increment hit count.
2. If not present, find if a page that is never referenced in future. If such a page exists, replace this page with new page. If no such page exists, find a page that is referenced farthest in future. Replace this page with new page.

Let's consider some examples:

Example 1:

Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2		
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

3. Least Frequently Used

The least frequently used (LFU) page replacement algorithm replaces the page in memory that has been accessed the fewest number of times. It works by assigning a counter to each page, and the counter is incremented each time the page is referenced. When a page needs to be replaced, the page with the lowest counter value is chosen.

Page fault and hit ratio

- A page fault occurs when a program tries to access a memory page that is not currently loaded into RAM, requiring the operating system to fetch it from disk. A page fault is an interrupt that occurs when a program tries to access a page that is not present in the main memory.

The hit ratio is the proportion of memory accesses that are "hits," meaning the page is already in memory. A higher hit ratio indicates better performance, as fewer page faults mean less time spent retrieving data from slower storage.

Hit Ratio:

- The hit ratio is a measure of how often the requested page is already in memory.
- It's calculated as the number of page hits divided by the total number of page references.

a program makes 100 memory accesses and 90 of them are found in memory (page hits), the hit ratio is 90/100 or 90%. The page fault rate would be 10/100 or 10%.

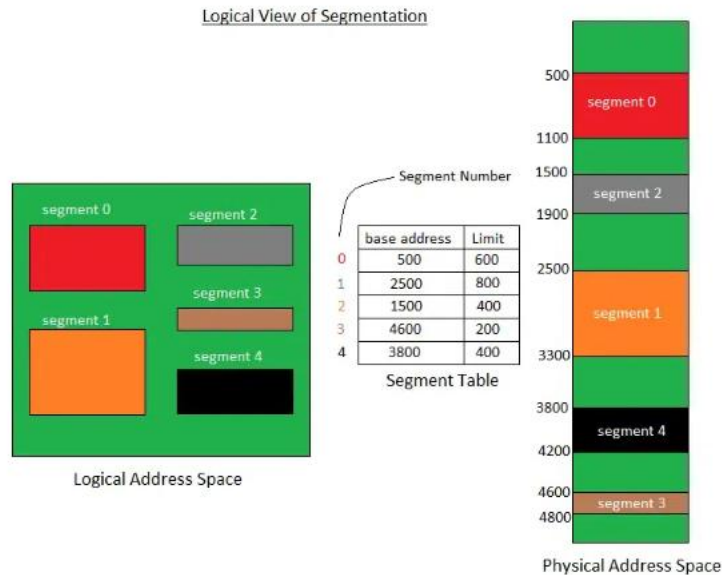
SEGMENTATION

A process is divided into Segments. The chunks that a program is divided into which are not necessarily all of the exact sizes are called segments. Segmentation gives the user's view of the process which paging does not provide. Here the user's view is mapped to physical memory.

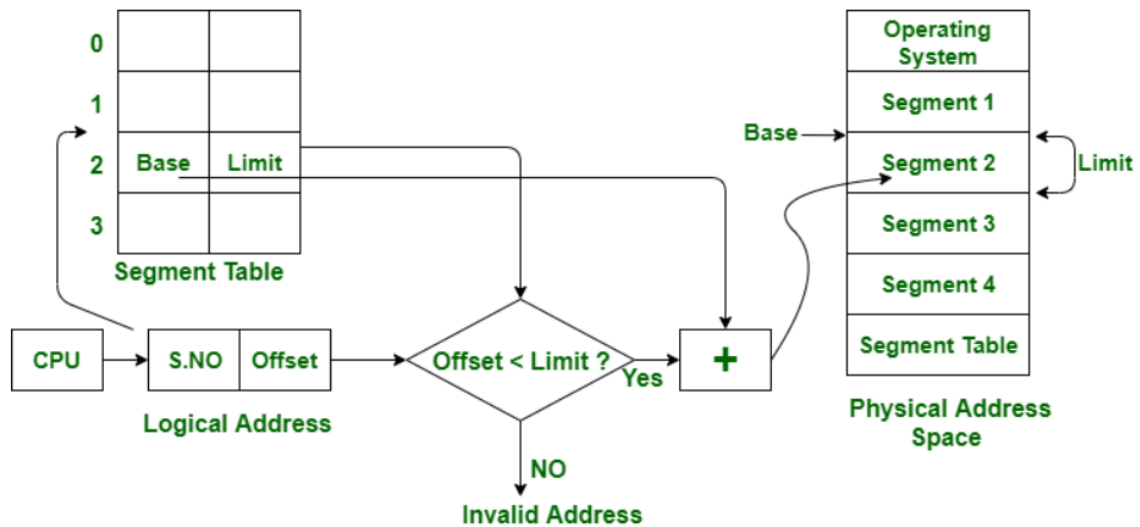
What is Segment Table?

It maps a two-dimensional Logical address into a one-dimensional Physical address. Its each table entry has:

- **Base Address:** It contains the starting physical address where the segments reside in memory.
- **Segment Limit:** Also known as segment offset. It specifies the length of the segment.



Segmentation



Difference Between Paging and Segmentation

Feature	Paging	Segmentation
Division Unit	Fixed-size pages	Variable-size segments
Managed By	Operating system	Compiler
Unit Size Determined By	Hardware	User/programmer
Data Structure Used	Page table	Segment table
Fragmentation Type	Internal fragmentation	External fragmentation
Speed	Faster	Slower

Solution of Round Robin scheduling algorithm of assessment.

Process	Arrival Time	CPU Burst Time
A	0	8
B	3	3
C	4	4
D	6	9

At t=0 ready queue [A]
 At t=2 ready queue [A]
 At t=4 ready queue [B,C,A]
 At t=6 ready queue [C,A,D,B]
 At t=8 ready queue [A, D,B,C]
 At t=10 ready queue [D,B,C,A]
 At t=12 ready queue [B,C,A,D]
 At t=13 ready queue [C, A, D, B]
 At t=15 ready queue [A, D, C]
 At t=17 ready queue [D]

A												
A	A											
A	A	B										
A	A	B	C									
A	A	B	C	A								
A	A	B	C	A	D							
A	A	B	C	A	D	B						
A	A	B	C	A	D	B	C					
A	A	B	C	A	D	B	C	A				
A	A	B	C	A	D	B	C	A	D	D	D	D

0 2 4 6 8 10 12 13 15 17 19 21 23 24

Assignment 1 : From Chapter 1

- ➔ POSIX standard
- ➔ Bootloader, MBR/GPT, UEFI vs Legacy Boot

Assignment 2: From Chapter 6

Even Roll No:

- ➔ OS security Principles:

a. Cryptography, multi-factor authentication (MFA), secure boot and sandboxing

➔ Access control Mechanism:

a. Policies, ACL, Capabilities and OS support

Odd Roll No:

➔ User Account Management and permissions

➔ System administration

a. Environment Configuration

b. Administrative tools : Shell Scripting, AWK, Make, CRON Jobs

Presentation from Chapter 8